



Синтез и анализ на алгоритми

за факултет

”Компютърни Системи и
Технологии”

проф. д-р Даниела Минковска

каб. 2324, кат. ПКТ, ФКСТ, ТУ-София

daniela@tu-sofia.bg



Синтез и анализ на алгоритми

ECTS кредити: 7

Хорариум:

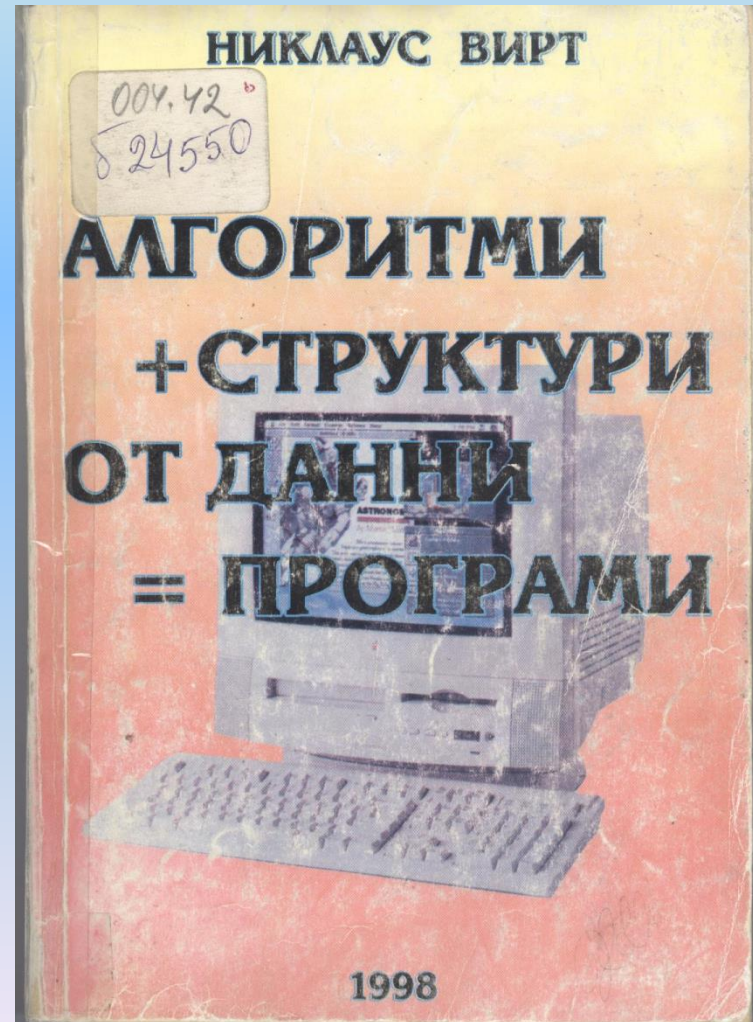
Лекции: **30 часа**

Лаб. Упражнения: **30 часа**

Курсов проект

Форма на контрол: *Изпит*

Основни учебни пособия





Списък с препоръчителна литература

1. Стойчо Д. Стойчев, Синтез и анализ на алгоритми, 2008, Авангард, Прима
2. Стойчев С., Касабов Н., Ръководство по синтез и анализ на алгоритми и програми, Техника, 1988
3. Уирт, Н., Алгоритми + структури от данни = програми, София, 1980
(Wirth, N. *Algorithms + Data Structures = Programs*, Prentice - Hall, New Jersey, 1976.)
4. Weiss, M. A. *Data Structures and Algorithm Analysis*, The Benjamin/Cummings Publishing Company, 1995.
5. Амерал Л. Алгоритми и структури от данни в C++. СОФТЕХ, София, 2001
6. Наков П., Добриков П., Програмиране = ++ Алгоритми. Top Team Co, 2002
7. Липский В., Комбинаторика для программистов. Мир, Москва, 1980
8. Clifford A. Shaffer, *Data Structures and Algorithm Analysis*, 2013
9. Sara Baase, *Computer Algorithms: Introduction to Design and Analysis*, 2009
10. Samir Khuler, *Design and Analysis of Algorithms*, 2012
11. A.A.Puntambekar, *Design and Analysis of Algorithms*, 2010
12. Седжуик Р., Алгоритми на С, том 1 – 4, СофтПрес, 2002.
13. Thomas Runkler, *Data Analytics*, 2012
14. Sedgewick R., Wayne K., *Algorithms*, Addison-Wesley Professional 2011
15. Записки от лекции



Сайт за новини и материали:

<http://pct.tu-sofia.bg/moodle001/>

Катедра "ПКТ"

- Състав на катедрата
- Контакти
- Проекти
- Електронна поща
- www.tu-sofia.bg

Програма "Еразъм+"

Съобщения за бакалаври

- Важно съобщение за бакалаври относно програмата "Еразъм +"

Съобщения за магистри

- Прием на магистри за учебната 2015-2016 година
- Заявление за кандидатстване в редовно платено обучение
- Заявление за кандидатстване в изравнително обучение по КТНС
- Заявление за кандидатстване в изравнително обучение по КТПП

- Важно съобщение за магистри относно програмата "Еразъм +"
- Официално откриване на учебната 2014/2015 година
- Програма за обмен на студенти - Еразъм
- Седмичен разпис за ОКС "Магистър", спец. КТПП
- Седмичен разпис за ОКС "Магистър", спец. КТНС
- Учебен план - изравнително обучение - КТПП
- Учебен план - изравнително обучение - КТНС
- Учебен план - КТПП

Programming and Computer Technologies



30 години катедра "Програмиране и компютърни технологии"

Обръщение на проф. д-р инж. Денчо Батанов, основател и първи ръководител на катедра ПКТ
Поздравителен адрес от Факултета по транспорта към ТУ-София
Поздравителен адрес от Машинно-технологичния факултет
Поздравителен адрес от Електротехнически факултет
Поздравителен адрес от катедра Електрически апарати

Категории курсове

Факултет Автоматика	1
Електротехнически факултет	7
Енергомашиностроителен факултет	1
Машинно-технологичен факултет	4
Машиностроителен факултет	4
Факултет Електронна техника и технологии	7
Факултет по телекомуникации	2
<u>Факултет Компютърни системи и управление</u>	

"Компютърни технологии и управление" - Компютърни: Computer Деактивиране

Пн	Вт	Ср	Чт	Пт	Сб	Нд
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29						

Предстоящи събития

Няма предстоящи събития

[Към календара...](#)

Последни новини

17 юни, 16:23
Admin Utilisateur
30 години катедра ПКТ още...
[Older topics ...](#)



Сайт за новини и материали:

<http://pct.tu-sofia.bg/moodle001/>

Катедра "Програмиране и компютърни технологии": Курсове Няма зареден профил. (Вход)

Катедра "Програмиране и компютърни технологии"

ПКТ ► Категории курсове ► ФКСУ (Бакалаври) Търсене на курсове: Go

Категории курсове:

Синтез и анализ на алгоритми
Преподавател: доц. д-р Даниела Минковска

Валидация и верификация на програмни системи
Преподавател: гл. ас. д-р Веселин Георгиев СЕМЕСТЪР VIII - 10 седмици

ПИК 1

ПИК 1
Преподавател: доц. д-р Динитър Божков

Катедра "Програмиране и компютърни технологии" Няма зареден профил. (Вход)

Катедра "Програмиране и компютърни технологии"

ПКТ ► Вход в сайта Български (bg) ▼

Връщате се на нашия сайт?

От тук можете да влезете в своя профил, използвайки своето потребителско име и парола (Вашият браузър трябва да приема бисквитки (cookies)) 🍪

Потребителско име
Парола

Някои курсове могат да се посещават от гости

Забравили сте потребителското си име или парола ?

За пръв път ли сте тук?

Please, ask your teacher for username and password.

Потребителско име: student
Парола: pktt



Лекция 1

Понятие за алгоритъм. Интуитивно понятие за алгоритъм - свойства, видове, характеристики. Машина на Тюринг. Понятие за програма и езици за програмиране.



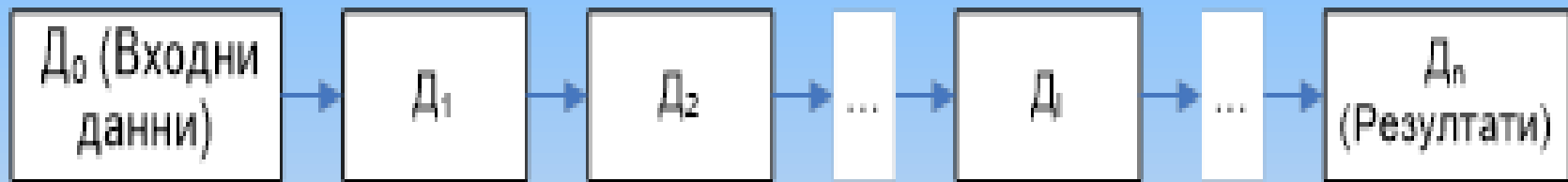
Основни понятия. Понятие за алгоритъм.

Алгоритъм – *интуитивно понятие*

1. Интуитивна дефиниция – последователност от инструкции (указания, стъпки, оператори) за действие, която при изпълнението си реализира зададена функционална зависимост между данните и резултатите.
2. Неформална дефиниция на Шнайдер и Герстинг (1995) – добре подредена колекция от еднозначни и ефективно изчислими операции, която след изпълнение дава резултат и спира изпълнението си за крайно време.

Основни понятия. Понятие за алгоритъм.

Алгоритъм - може да се разглежда като *преобразувател на данни* – при изпълнението си той преобразува данните от едно представяне в друго, докато се получат резултатите:



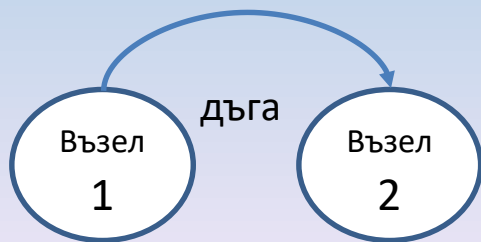
Алгоритмите заменят (разлагат) сложното действие (преобразуване на входните данни в резултати) на по-прости действия, които могат да се изпълнят от изпълнителя (компютър, специализирано устройство, човек).

Основни понятия. Понятие за алгоритъм.

Във всяка инструкция се указват всички или някои от следните елементи:

- 1) Каква операция/операции се извършват?
- 2) Какви са нейните аргументи или резултати от изпълнението?
- 3) Коя е следващата инструкция за изпълнение?

Алгоритъмът Може да се представи и като ориентиран граф (с възли и дъги)



възлите – операциите;

дъгите – преходите, показващи кои могат да са следващите операции (възли) в алгоритъма



Основни понятия. Примери на известни алгоритми

- 1) Начини за извършване на аритметични действия („+“, „-“, „*“, „/“);
- 2) Числено решаване на с-ми линейни уравнения (напр. Метод на Гаус, на простата итерация, итерационен метод на Зайдел и др.);
- 3) Числени методи за изчисляване на интеграл (метод на трапеците, на Симпсон и др.);
- 4) Числени методи за решаване на диференциални уравнения (метод на Рунге-Кута, на Адамс и др.);



Основни понятия. Примери на известни алгоритми

- 5) Анализ на логически схеми и крайни автомати;
- 6) Анализ на електрически схеми (по методите на възловите потенциали и контурните токове);
- 7) Алгоритъм за обработка на машинен детайл с металорежеща машина с цифрово управление;



Свойства на алгоритмите

Свойства на формалните алгоритми:

- а) определеност (детерминираност);
- б) крайност (финитност);
- в) дискретност;
- г) допускат указания и действия над входни данни (дефиниционна област);
- д) резултатност;
- е) масовост- приложимост за различни начални стойности на данните от определено множество;
- ж) цикличност – притежават го само някои алгоритми.



Свойства на алгоритмите

1. Определеност (детерминираност) – при едни и същи данни при различни изпълнения, алгоритъмът като цяло (както и неговите отделни стъпки) дават *един и същи точно определен резултат* (това не важи за вероятностните алгоритми, т.к. при тях има поне една стъпка със случаен избор);

2. Крайност (финитност) – Алгоритъмът и всяка негова стъпка се изпълняват за *крайно време*. Алгоритъмът съдържа краен брой стъпки и следователно се изпълнява за краен брой стъпки.

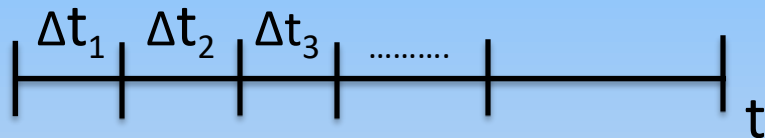
(Изключение: Задача на Колац /нерешима задача/ - пример за последователност от инструкции, за които не е доказано, че е Алгоритъм, т.е. тя не завършва изпълнението си за крайно време).



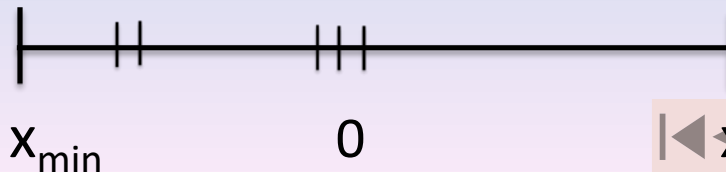
Свойства на алгоритмите

3. Дискретност – изпълнението на алгоритъма във времето се извършва на **интервали (стъпки)**. Този процес е дискретен, но е непрекъснат. Има 2 вида дискретност:

а) *дискретност във времето* – следващата стъпка Δt_2 не може да започне преди Δt_1 да е завършила.



б) *дискретност в представянето на числата* – всеки процесор работи с краен брой числа в определен диапазон ($x_{\min}$ до $x_{\max}$), който зависи от дължината на машинната дума. Между две реални числа има много числа, но не всички могат да се представят, и това води до грешки в представянето на числа в определен интервал.





Свойства на алгоритмите

4. Множество от входни данни (дефиниционна област /ДО/) – данни за които **алгоритъмът работи коректно**.

Напр. Алгоритъмът на Евклид работи само с цели числа (реалните не са в неговата ДО). В частни случаи ДО не може да се определи лесно, напр. в итерационни алгоритми за решаване на с-ми от уравнения, т.к. решението им се уточнява постепенно.

5. Резултатност – верни ли са алгоритмите? 2 начина за проверка на коректност:

- а) експериментален – тестов набор и известни резултати;
- б) аналитичен – подават се на входа вх. данни (покриващи ДО), на изхода – изх. условия, и във вътрешни точки – междинни условия. Ако се получат зададените изх. условия, значи алг. е верен.

Празно множество от резултати не е допустимо!



Описание на алгоритмите

- 1. Словесно** – чрез изречения на естествен език и означения от съответната област. *Недостатък: може лесно да се получи нееднозначност!*
- 2. Чрез блок - схема** – много удобен и нагледен метод за представяне на участъци от алгоритъмът със сложна логика (напр. разклонения, цикли). *Недостатък: заема много място!*
- 3. Чрез език за програмиране** – програма, напр. C, C++ и др. *Не допуска нееднозначност!*
- 4. С теоритични алгоритмични системи** – те съдържат макс. прости инструкции. Служат за теоритични изследвания дали зададена задача има алг. за нейното решаване – ако „да“ - се нарича „**алгоритмично решаема**“, ако „не“ – „**алгоритмично нерешима**“. Напр. (машина на Тюринг, система на Пост, алгоритъм на Марков, рекурсивни функции и др.)



Машина на Тюринг

Машина на Тюринг (MT) е абстрактно изчислително устройство, описано от английския математик Алън Тюринг през 1936 г.

MT е автомат, обработващ безкрайна лента, разделена на участъци (клетки), на която се записва обработваната информация. Наборът от допустимите символи за запис върху лентата за дадена машина на Тюринг е краен и се нарича **външна азбука**. Един от символите на външната азбука е задължителен – празна позиция (означена с Δ).

Във всяка една клетка може да бъде записан (прочетен) един символ:

$$a_0, a_1, a_2, \dots, a_n$$

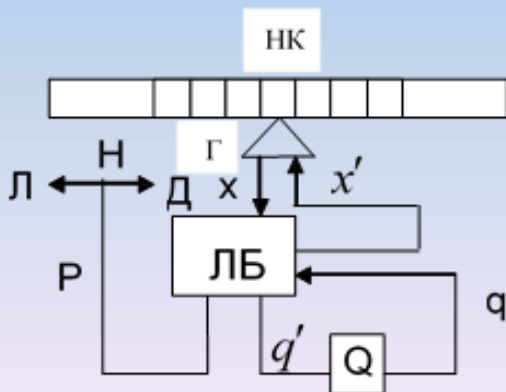


Машина на Тюринг

Устройство на МТ: Съществуват варианти с безкрайна лента в едната посока и с безкрайна лента в двете посоки. Лентата се състои от клетки, във всяка от които има по един символ. Една от тези клетки се нарича наблюдавана (НК) и към нея има свързана глава, която може да чете символ от НК и да записва символ в НК. Освен това МТ има логически блок (ЛБ) и блок на състоянията (Q). Множеството от символите на блока на състоянията се нарича вътрешна азбука.

ЛБ – логически блок;

Q - блок на състоянията;



ЛБ има два входа : прочетения символ x и текущото състояние q . В резултат ЛБ изработва 3 изхода :

x' – изтрива се от наблюдаваната клетка стария символ x и се записва новия x' .

q' - е новото състояние.

P - изход за преместване на главата.



Машина на Тюринг

Принцип на работа на МТ: за предвижване на лентата МТ разполага с прост механизъм, който:

- предвижва лентата с една стъпка напред;
- Предвижва лентата с една стъпка назад;
- оставя лентата неподвижна.

Т. е. лентата може да се намира в едно от трите състояния:

$$d_{-1} \ d_{+1} \ d_0$$

Действието на машината на Тюринг протича на тактове. При всеки такт се изпълнява една команда, която има вида $xq \rightarrow x'Rq'$, където x е прочетения символ от НК, q е текущото състояние, x' е символът, който се записва в НК, при което x се изтрива, q' е новото състояние, а R показва къде ще се намира главата при следващия такт на изпълнение.



Машина на Тюринг

Принцип на работа на МТ: Множеството от всички команди на МТ се записва в т.нар. **функционална таблица** (или функционална схема), която се изпълнява от логическия блок (ЛБ). Всеки неин ред съответства на символ от външната азбука и всеки неин стълб – на символа на едно вътрешно състояние.

вътрешни състояния

	вътрешни състояния				
	q_1	q_2	q_3	$\dots$	q_m
външна азбука	X_1	$X_{11}p_{11}q_{11}$	$X_{12}p_{12}q_{12}$	$X_{13}p_{13}q_{13}$	$X_{1m}p_{1m}q_{1m}$
	X_2	$X_{21}p_{21}q_{21}$	$X_{22}p_{22}q_{22}$	$X_{23}p_{23}q_{23}$	$X_{2m}p_{2m}q_{2m}$
	X_3	$X_{31}p_{31}q_{31}$	$X_{32}p_{32}q_{32}$	$X_{33}p_{33}q_{33}$	$X_{3m}p_{3m}q_{3m}$
	$\vdots$				
	X_n	$X_{n1}p_{n1}q_{n1}$	$X_{n2}p_{n2}q_{n2}$	$X_{n3}p_{n3}q_{n3}$	$X_{nm}p_{nm}q_{nm}$

В клетката на функционалната таблица от ред x и стълб q се записва тройка символи $x'Pq'$, т.е. всяка клетка от таблицата съответствува на една команда на МТ.

Функционалната таблица се нарича още тюрингова програма.



Средства за представяне на алгоритмите

Средства за представяне на алгоритмите и основни команди за описване на алгоритми:

- за присвояване на стойност;
- за въвеждане и извеждане;
- за избор на вариант (условни команди);
- за организиране на цикъл;
- за обръщение към подалгоритъм.



Класификация (варианти) на алгоритмите

1. Според структурата на алгоритъма;
2. Според броя инструкции, които се изпълняват едновременно;
3. Според спазването на условието за детерминираност;
4. Според оптималността на резултатите;
5. Според възможността за самообучение;
6. Числени и нечислени.



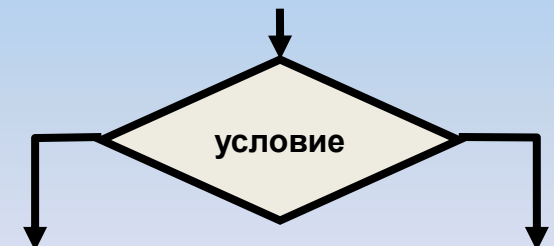
Класификация (варианти) на алгоритмите

1. Според структурата на алгоритъма:

а) *линейни* - инструкциите се изпълняват в строга последователност една след друга, независимо от резултатите, получени след изпълнението на предходните инструкции; *срещат се рядко!*

б) *разклонени* - действията продължават с изпълнението на процес, избран от набор взаимно изключващи се възможни процеси; Биват 2 вида:

- циклични – повтарят се краен брой пъти докато се изпълни дадено условие;
- нециклични





Класификация (варианти) на алгоритмите

2. Според броя инструкции, които се изпълняват едновременно – биват 2 вида:

а) *последователни* – в даден момент може да се изпълнява **само една инструкция** на алгоритъма.

б) *паралелни* – в даден момент могат да се изпълняват **повече от една инструкции**. Изпълняват се в многопроцесорните (8,16,32,64,128 и т.н.) компютри.

3. Според спазването на условието за детерминираност:

а) *детерминирани*;

б) *недетерминирани* (вероятностни).

4. Според възможността за самообучение:

а) *самообучаващи се*;

б) *несамообучаващи се*.



Класификация (варианти) на алгоритмите

5. Според оптималността на резултатите:

- а) *точни* – алг., който **винаги дава оптимален резултат**;
- б) *евристични (приблизителни)* – дават **резултат, близък до оптималния**, или въобще не дават резултат.

Използват се при:

- ако съществува алгоритъм за решаване на дадена задача, но той е толкова сложен, че не може да бъде съставен;
- ако съществува алгоритъм, но той се изпълнява много бавно или за не приемливо време (месеци, години);
- ако не стига наличната памет на компютъра.

6. Числени и нечислени:

- а) *числени* – основни са аритметичните операции;
- б) *нечислени* – обработват предимно символна инф-я.



Примерен алгоритъм: *Определяне на НГОД на две цели числа m и n*

Алг. 1: Най-прост и най-неефективен:

- а) взима се по-малкото число и се дели до 2;
- б) намира се най-голямото число на което се дели (от естествените числа до 9);
- в) повтаря се същото и с другото число;

Най-високият общ делител се нарича НГОД!

Примери:

-НГОД (45,18)

18/2, 18/3, 18/6, 18/9

45/3, 45/5, 45/9



НГОД (45,18) = 9

-НГОД (21,35) = 7

-НГОД (80,36) = 4



Примерен алгоритъм: *Определяне на НГОД на две цели числа m и n*

Алг. 2: Алгоритъм на Евклид: *Анализ на задачата и се определят свойствата ѝ с цел да се използват в алгоритъма:*

а) $\text{НГОД}(m, 0) = m$. Ако и $m=0 \implies \text{НГОД}(0, 0) = 0$

б) $\text{НГОД}(-m, n) = \text{НГОД}(m, n) \implies$ Можем да работим само с положителни числа.

в) $\text{НГОД}(m, n) = \text{НГОД}(n, r)$, където r е остатъкът от делението на m с n , т.е. $M = q \cdot n + r$, $0 \leq r \leq n$

(q – целочисленият резултат).



Примерен алгоритъм: *Определяне на НГОД на две цели числа m и n*

Алг. 2: Алгоритъм на Евклид: Пример: Нека $m=21$, $n=35$

N: итерация	m	n	q	r	обяснение
0	21	35	0	21	$21:35=0 \rightarrow q=0$; r :=по-малкото число = 21
1	35	21	1	14	$m:=n$; $n:=r$; $35:21=1$ q 21 ----- 14 r
2	21	14	1	7	$m:=n(21)$; $n:=r(14)$; $21:14=1$ q 14 ----- 7 r
3	14	7	2	0	$m:=n(14)$; $n:=r(7)$; $14:7=2$ q 14 ----- 0 r
4	7	0	0	0	$m:=n(7)$; $n:=r(0)$; $7:0=0$ q 0 ----- 0 r

За всяка итерация за m

$\Rightarrow m = q \cdot n + r$, т.е

0: $m = 0 \cdot 35 + 21 = 21$

1: $m = 1 \cdot 21 + 14 = 35$

2: $m = 2 \cdot 7 + 0 = 14$

3: $m = 1 \cdot 7 + 0 = 7$

4: $m = 0 \cdot 0 + 0 = 0$

$\Rightarrow \text{НГОД}(m, n) = m, \Rightarrow \text{НГОД}(21, 35) = 7$



Примери за описание на алгоритъмът на Евклид.

1. **Словесно:** Ако $C1, C2$ и $C3$ са номерата на инструкциите (стъпките), следователно:

→ **$C1$:** Ако $n=0$, то НГОД $(m,n)=m$, и **край!**

ако $n \neq 0$, то $\rightarrow$ преход към $C2$

$C2$: Дели се m на $n \rightarrow$ получава се остатък r

$m:=n$;

$n:=r$; (т.е. **m и n получават нови стойности**)

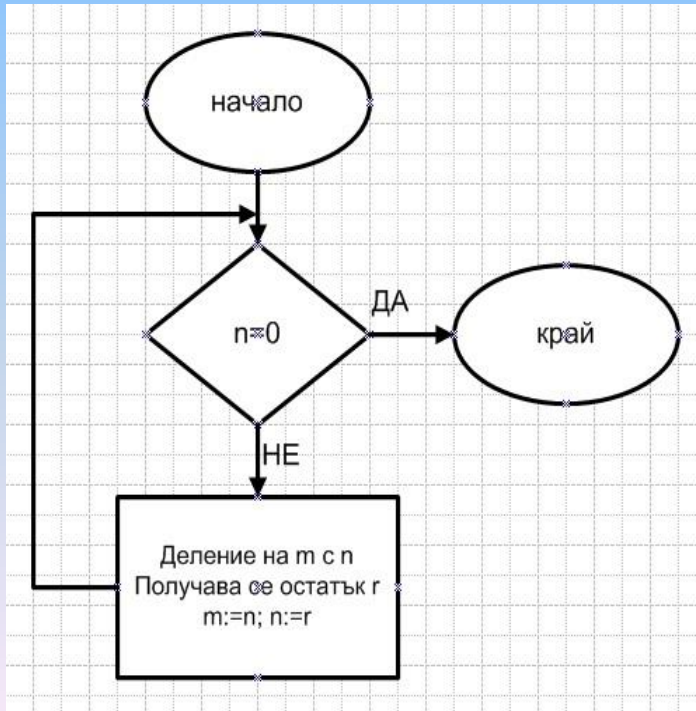
$C3$: премини към стъпка $C1$.

Недостатък: Може лесно да се получи **нееднозначност!**

Примери за описание на алгоритъмът на Евклид.

2. Чрез блок – схема:

Много нагледен и удобен начин за представяне на участъци със сложна логика. Недостатък: заема много място!



3. Чрез език за програмиране:

(*не допуска нееднозначност!*)

unsigned got (unsigned a, unsigned b)

{

unsigned temp;

while (b>0)

{

temp = b;

b = a % b;

a = temp;

}

return a;

}



Понятие за програма и езици за програмиране

Алгоритми + структури от данни = програми

Програма (*Н. Уирт*) – конкретни формулировки на абстрактни алгоритми, които се базират на определено представяне и структури на данни.

С език за програмиране се кодират алгоритмите и структурите от данни в програмата.

Класификация на програмните езици:

- Езици от ниско ниво – машинен и асемблерен;
- Езици от високо ниво:
 - Императивни езици (структурирани) – изразяващи команда – BASIC, FORTRAN, PASCAL, C/C++/C#, Ada, Modula-2, Java, Python;
 - Приложни езици – специализирани за решаване на един вид проблем – езици за бази данни (SQL), за моделиране (VHDL), за мултимедийни приложения (HTML, OpenScript, Lingo и др.)



Етапи на съставяне и изпълнение на програма

